

# Changes between C++17 and C++20 DIS

## Abstract

This document enumerates all the major changes that have been applied to the C++ working draft since the publication of C++17, up to the publication of the C++20 DIS. Major changes are those that were added in the form of a dedicated paper, although not all papers are mentioned individually. The remaining papers are listed summarily below. Issue resolutions from the CWG or LWG issues lists are generally not included, but the containing papers are part of the summary listing. Defect reports that are not part of the published International Standard are marked with a “DR” superscript.

## Contents

1. [Removed or deprecated features](#)
2. [New core language features with global applicability](#)
3. [New core language features with local applicability](#)
4. [New library features](#)
5. [Modifications to existing features](#)
6. [Miscellaneous](#)
7. [Unlisted papers](#)
8. [Defects, issues, bug fixes](#)
9. [Assorted snippets demonstrating C++17](#)

## Removed or deprecated features

| Document                | Summary                                | Examples, notes                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-------------------------|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">P0619R4</a> | Removal of various deprecated features | The following features that were already deprecated in C++17 are now removed: <ul style="list-style-type: none"><li>• the <i>noexcept-specifier</i> <code>throw()</code> (use <code>noexcept</code> instead)</li><li>• ineffective “C++ versions” of compatibility headers, <code>&lt;ccomplex&gt;</code>, <code>&lt;ciso646&gt;</code>, <code>&lt;cstdalig&gt;</code>, <code>&lt;cstdbool&gt;</code>, <code>&lt;ctgmath&gt;</code> (there is no need for those in C++ code)</li></ul> |

| Document | Summary                                                  | Examples, notes                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|----------|----------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|          |                                                          | <ul style="list-style-type: none"> <li>• <code>uncaught_exception</code> (use <code>uncaught_exceptions</code> instead)</li> <li>• functional adaptors <code>not1</code>, <code>not2</code>, <code>unary_negate</code>, <code>binary_negate</code>, as well as the class members <code>result_type</code>, <code>argument_type</code>, <code>first_argument_type</code>, <code>second_argument_type</code> (the functionality of <code>bind</code> and <code>not_fn</code> is strictly superior).</li> <li>• redundant members of <code>allocator</code> (which are subsumed by the <code>allocator_traits</code> defaults): <code>pointer</code>, <code>const_pointer</code>, <code>reference</code>, <code>const_reference</code>, <code>rebind</code>, <code>address</code>, <code>construct</code>, <code>destroy</code>.</li> <li>• <code>raw_memory_iterator</code></li> <li>• temporary buffer API (<code>get_temporary_buffer</code>, <code>return_temporary_buffer</code>)</li> <li>• traits <code>is_literal_type</code> and <code>result_of</code></li> <li>• the member <code>shared_ptr::unique</code></li> </ul> <p>Moreover, formerly implied special member functions are now declared explicitly throughout the standard library whenever their implicit declaration is deprecated [depr.impldec].</p> |
| P1236R1  | Only two's complement representation for signed integers | As of C++20, the only permitted representation for signed integers is two's complement. Previously, no representation was specified, but due to size and layout constraints, the only possible ones were two's complement, ones' complement, and sign-magnitude. Support for the latter two has now been removed.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| P0767R1  | Deprecate the notion of a "POD" type                     | The notion of a "plain-old data" (POD) type has been superseded by the more granular notions of <i>trivial</i> and <i>standard-layout</i> , and this change removes this notion from the main language specification (though it is retained as deprecated in Annex D).                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |

| Document                                             | Summary                                                                         | Examples, notes                                                                                                                                                                                                                                                                                                                    |
|------------------------------------------------------|---------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">P0806R2</a>                              | Deprecate the implicit lambda capture of <code>this</code> via <code>[=]</code> | The capture of <code>this</code> via <code>[=]</code> is now deprecated, since it is irregular and potentially confusing with <code>[=, *this]</code> . The capture should now be spelled <code>[=, this]</code> . See also P0409R2 below.                                                                                         |
| <a href="#">P1161R3</a>                              | Deprecate the comma operator in subscripting expressions                        | The syntax <code>a[x, y, z]</code> is now deprecated; if this functionality is desired, it should be spelled <code>a[(x, y, z)]</code> . Eventual removal of the deprecated expression would open the door to extensions to the subscripting expression.                                                                           |
| <a href="#">P1152R4</a> ,<br><a href="#">P1831R1</a> | Deprecate certain <code>volatile</code> qualifications                          | The <code>volatile</code> qualifications in certain contexts where they would be of questionable value or outright broken are now deprecated. (This includes certain operands of increment/decrement and assignment expressions, function parameters, function return types, and the types of structured bindings.)                |
| <a href="#">P0966R1</a>                              | Deprecate shrinking <code>basic_string::reserve</code>                          | The default argument of the parameter of <code>basic_string::reserve</code> is removed and the function no longer shrinks capacity. The behaviour was inconsistent with <code>vector::reserve</code> and redundant with <code>basic_string::shrink_to_fit</code> . A deprecated zero-parameter overload has been added to Annex D. |

## New core language features with global applicability

These are features that may happen to you without your knowledge or consent.

| Document                                                                                                                                                                            | Summary  | Examples, notes                                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Core:<br><a href="#">P0734R0</a> ,<br><a href="#">P0857R0</a> ,<br><a href="#">P1084R2</a> ,<br><a href="#">P1141R2</a> ,<br><a href="#">P1452R2</a> ,<br><a href="#">P1616R1</a> , | Concepts | Concepts extend C++'s template facility with a way to constrain a template's parameters. The constraints take part in overload resolution. |

| Document                                                                                                                                                                                                                                                                                                                                                             | Summary                                                      | Examples, notes                                                                                                                                                                                                                                                                                                                       |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">P1972R0</a> ,<br><a href="#">P1980R0</a> ,<br><a href="#">P2092R0</a> ,<br><a href="#">P2104R0</a> ,<br><a href="#">P2113R0</a> ;<br>Library:<br><a href="#">P0898R3</a> ,<br><a href="#">P1248R1</a> ,<br><a href="#">P1474R1</a> ,<br><a href="#">P1754R1</a> ,<br><a href="#">P1878R1</a> ,<br><a href="#">P1964R2</a> ,<br><a href="#">P2102R0</a> . |                                                              |                                                                                                                                                                                                                                                                                                                                       |
| <a href="#">P0912R5</a> ,<br><a href="#">P2107R0</a>                                                                                                                                                                                                                                                                                                                 | Coroutines                                                   | Coroutines are a new kind of control flow that extends the notion of a function.                                                                                                                                                                                                                                                      |
| <a href="#">P1103R3</a> ,<br><a href="#">P1502R1</a> ,<br><a href="#">P1766R1</a> ,<br><a href="#">P1811R0</a> ,<br><a href="#">P1815R2</a> ,<br><a href="#">P1703R1</a> ,<br><a href="#">P1779R3</a> ,<br><a href="#">P1857R3</a> ,<br><a href="#">P1874R1</a> ,<br><a href="#">P1979R0</a> ,<br><a href="#">P2109R0</a> ,<br><a href="#">P2115R0</a>               | Modules                                                      | Modules offer a new way to organize source code that is not based on textual inclusion in the way header inclusions were, but instead allows for a principled specification of component boundaries.                                                                                                                                  |
| <a href="#">P0515R3</a> ,<br><a href="#">P0905R1</a> ,<br><a href="#">P1120R0</a> ,<br><a href="#">P1185R2</a> ,<br><a href="#">P1186R3</a> ,<br><a href="#">P1630R1</a> ,<br><a href="#">P1614R2</a> ,<br><a href="#">P1855R0</a> ,<br><a href="#">P1946R0</a> ,<br><a href="#">P1959R0</a> ,<br><a href="#">P2002R1</a> ,<br><a href="#">P2085R0</a>               | Three-way comparison (“spaceship”) and defaulted comparisons | <p>The value of the expression <code>a &lt;=&gt; b</code> describes which one of <code>a &lt; b</code>, <code>a == b</code>, and <code>a &gt; b</code> is true.</p> <p>A key part of this feature is that classes that provide this operator can have all of the six traditional, binary relational operators defined implicitly.</p> |

# New core language features with local applicability

These are features where you would know if you were using them.

| Document                                             | Summary                                                | Examples, notes                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|------------------------------------------------------|--------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">P0329R4</a>                              | Designated initializers                                | <code>struct X { int a; int b; }; X x { .b = 10 };</code>                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <a href="#">P0732R2</a> ,<br><a href="#">P1907R1</a> | Class-types in non-type template parameters            | Non-type template parameters can now be of class type: <code>struct X { /* ... */};</code><br><code>template &lt;X Value&gt; struct Y;</code> Certain restrictions apply to ensure that value identity can be established reliably. These new kinds of non-type parameters are lvalues, unlike the existing non-class ones.                                                                                                                                                     |
| <a href="#">P0840R2</a>                              | <code>[[no_unique_address]]</code>                     | An attribute that provides language support for empty or overlapping member objects. Previously, this effect was often achieved by using overlapping base class layouts (“empty base optimisation”).                                                                                                                                                                                                                                                                            |
| <a href="#">P0479R5</a>                              | <code>[[likely]]</code> ,<br><code>[[unlikely]]</code> | Attributes to guide branch prediction. Already wide-spread as vendor extensions.                                                                                                                                                                                                                                                                                                                                                                                                |
| <a href="#">P1007R3</a>                              | <code>assume_aligned</code>                            | A special language-support function that promises to the implementation that a pointer is aligned in a certain way (with undefined behaviour if this is not in fact true at runtime), which allows better code generation (e.g. vectorised instructions; omission of preambles). Already wide-spread as vendor extensions.                                                                                                                                                      |
| <a href="#">P1143R2</a>                              | <code>constinit</code>                                 | The new <code>constinit</code> keyword specifies a variable to be constant-initialized (and the program would be ill-formed if this is not possible). The variable can still be mutable, and indeed this is the key difference from a <code>constexpr</code> variable. Having constant initialization suffices to avoid any form of static initialization problems, and also allows local static variables to omit a synchronization guard on access (since there is no dynamic |

| Document            | Summary                                   | Examples, notes                                                                                                                                                                                                                                                                                                                                                                                               |
|---------------------|-------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                     |                                           | initialization that any thread would need to wait for).                                                                                                                                                                                                                                                                                                                                                       |
| P1073R3,<br>P1937R2 | constexpr                                 | The new <code>constexpr</code> keyword specifies a function as an “immediate function”, whose invocation is always a constant expression (and which thus never requires code to be emitted from the translation). Contrast this with <code>constexpr</code> functions, which <i>may</i> , but are not required to be, used as constant expressions.                                                           |
| P0595R2             | is_constant_evaluated                     | A special language-support function that tells whether the caller is in a constant evaluation. This allows library implementations to provide a <code>constexpr</code> -implementation as well as an optimised runtime implementation of the same function (whereas the <code>constexpr</code> implementation alone would not have optimal runtime performance, if it turned out to be needed at runtime).    |
| P0722R3             | A destroying operator <code>delete</code> | A new way to provide a deallocation function ( <code>operator delete</code> , taking a parameter of type <code>std::destroying_delete_t</code> ) with the effect that a <i>delete-expression</i> that would choose such a function does <i>not</i> call the destructor; instead, the deallocation function itself is required to call the destructor. See the paper for examples why this might be desirable. |
| P0784R7             | Constexpr <code>new/delete</code>         | Dynamic object creation and destruction may now appear in <code>constexpr</code> contexts, as long as every created object is destroyed. This allows, say, the use of a local <code>std::string</code> variable in the body of a <code>constexpr</code> function.                                                                                                                                             |
| P0476R2             | bit_cast                                  | Despite its library-looking name, this is a core language facility that creates objects whose object representation is copied from an existing object. Importantly, the target type need not be default-constructible. Attempts to achieve this by other means, while very common, often run into undefined behaviour.                                                                                        |

| Document                | Summary    | Examples, notes                                                                   |
|-------------------------|------------|-----------------------------------------------------------------------------------|
|                         |            | Example: <code>std::uint32_t n = std::bit_cast&lt;std::uint32_t&gt;(1.5f);</code> |
| <a href="#">P1099R5</a> | using enum | A way to introduce the enumerator names of a scoped enumeration.                  |

## New library features

| Document                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | Summary                  | Examples, notes                                                                                                                                                                                                                 |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">P0896R4</a> ,<br><a href="#">P1252R2</a> ,<br><a href="#">P1035R7</a> ,<br><a href="#">P1243R4</a> ,<br><a href="#">P1391R4</a> ,<br><a href="#">P1394R4</a> ,<br><a href="#">P1456R1</a> ,<br><a href="#">P1523R1</a> ,<br><a href="#">P1716R3</a> ,<br><a href="#">P1739R4</a> ,<br><a href="#">P1862R1</a> ,<br><a href="#">P1870R1</a> ,<br><a href="#">P1871R1</a> ,<br><a href="#">P1970R2</a> ,<br><a href="#">P1983R0</a> ,<br><a href="#">P1994R1</a> ,<br><a href="#">P2091R0</a> ,<br><a href="#">P2106R0</a> | Ranges                   | Ranges are a new model that improves on pairs of iterators. This feature consists of new taxonomy, new concepts, and new versions of the familiar standard library algorithms that live in namespace <code>std::ranges</code> . |
| <a href="#">P0355R7</a> ,<br><a href="#">P1361R2</a> ,<br><a href="#">P1466R3</a> ,<br><a href="#">P1650R0</a> ,<br><a href="#">P1981R0</a> ,<br><a href="#">P1982R0</a>                                                                                                                                                                                                                                                                                                                                                             | Calendars and time zones | A large library addition to <code>&lt;chrono&gt;</code> for civil time (calendars). For example: <code>auto date = 2016y/May/29;</code>                                                                                         |
| <a href="#">P0645R10</a> ,<br><a href="#">P1361R2</a> ,<br><a href="#">P1652R1</a> ,<br><a href="#">P1868R2</a> ,<br><a href="#">P1892R1</a>                                                                                                                                                                                                                                                                                                                                                                                         | Text formatting          | <code>std::string s = std::format("{} ", "Hello, world");</code>                                                                                                                                                                |

| Document                                                                                                                                                                                                                           | Summary                                                     | Examples, notes                                                                                                                                                                                                                                                                                                                                                                                                                                |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">P1135R6</a> ,<br><a href="#">P1643R1</a> ,<br><a href="#">P1644R0</a> ,<br><a href="#">P1865R1</a>                                                                                                                     | Synchronization                                             | Waiting, notifications, semaphores, latches, barriers.                                                                                                                                                                                                                                                                                                                                                                                         |
| <a href="#">P0631R8</a>                                                                                                                                                                                                            | Mathematical constants                                      | Variable templates for many mathematical constants, e.g. <code>std::numbers::pi_v&lt;float&gt;</code> , and concrete constants for <code>double</code> , e.g. <code>std::numbers::pi</code> .                                                                                                                                                                                                                                                  |
| <a href="#">P1208R6</a>                                                                                                                                                                                                            | <code>source_location</code>                                | A proper library type that exposes the information provided by the <code>__LINE__</code> and <code>__FILE__</code> macros, the <code>__func__</code> variable, and column number.                                                                                                                                                                                                                                                              |
| <a href="#">P0122R7</a> ,<br><a href="#">P1085R2</a> ,<br><a href="#">P1024R3</a> ,<br><a href="#">P1227R2</a> ,<br><a href="#">P1394R4</a> ,<br><a href="#">P1872R0</a> ,<br><a href="#">P1976R2</a> ,<br><a href="#">P2116R0</a> | New class template: <code>span</code>                       | A <code>span</code> is a light-weight reference to a contiguous range (mutable or const); essentially a “pointer and size”: A <code>span&lt;T&gt;</code> is like a <code>T* p</code> and a size <code>N</code> such that <code>p[i]</code> is valid for <code>i</code> in <code>[0, N)</code> . It is similar in purpose (and pitfalls) to C++17’s <code>string_view</code> , which is a special kind of <code>span</code> adapted to strings. |
| <a href="#">P0660R10</a> ,<br><a href="#">P1869R1</a>                                                                                                                                                                              | New classes: <code>stop_token</code> , <code>jthread</code> | Mechanisms for a thread manager class that joins the thread on destruction, and for interrupting the running thread.                                                                                                                                                                                                                                                                                                                           |
| <a href="#">P0019R8</a> ,<br><a href="#">P1123R0</a>                                                                                                                                                                               | New class template: <code>atomic_ref</code>                 | Atomic operations on non-atomic objects.                                                                                                                                                                                                                                                                                                                                                                                                       |
| <a href="#">P0550R2</a> ,<br><a href="#">P0777R1</a>                                                                                                                                                                               | Transformation trait: <code>remove_cvref</code>             | A composition of <code>remove_cv</code> and <code>remove_reference</code> . Previously, the slightly different <code>decay</code> had been popular for this use case, even though it does too much. Existing uses of <code>decay</code> in the specification have been replaced with the more appropriate transformations.                                                                                                                     |

| Document                                             | Summary                                                                                                                                                                                                        | Examples, notes                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">P0318R1</a>                              | Transformation traits:<br><a href="#">unwrap_reference</a> , <a href="#">unwrap_ref_decay</a>                                                                                                                  | Traits to work with <a href="#">reference_wrapper</a> :<br><a href="#">unwrap_reference_t&lt;T&gt;</a> is <a href="#">U&amp;</a> if <a href="#">T</a> is <a href="#">reference_wrapper&lt;U&gt;</a> , and <a href="#">T</a> otherwise. <a href="#">unwrap_ref_decay</a> first applies <a href="#">decay</a> before unwrapping. The motivating use case is the specification of <a href="#">make_pair</a> ; these traits allow users to write functions with analogous handling of <a href="#">reference_wrapper</a> . |
| <a href="#">P0887R1</a>                              | Transformation trait: <a href="#">type_identity</a>                                                                                                                                                            | <a href="#">type_identity_t&lt;T&gt;</a> is <a href="#">T</a> ; useful when a template parameter is required in a non-deduced context.                                                                                                                                                                                                                                                                                                                                                                                |
| <a href="#">P0758R1</a>                              | Type trait: <a href="#">is_nothrow_convertible</a>                                                                                                                                                             | Like <a href="#">is_convertible</a> , but does not throw.                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| <a href="#">P1357R1</a>                              | Type traits: <a href="#">is_bounded_array</a> ,<br><a href="#">is_unbounded_array</a>                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <a href="#">P0466R5</a>                              | Type traits: <a href="#">is_layout_compatible</a> ,<br><a href="#">is_pointer_interconvertible_base_of</a> ,<br><a href="#">is_pointer_interconvertible_class</a> ,<br><a href="#">is_corresponding_member</a> |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <a href="#">P1115R3</a> ,<br><a href="#">P1209R0</a> | Free functions <a href="#">erase</a> , <a href="#">erase_if</a>                                                                                                                                                | Free functions to provide a simple, uniform interface for erasing elements from a container. The functions return the number of erased elements; see also <a href="#">P0646R1</a> below.                                                                                                                                                                                                                                                                                                                              |
| <a href="#">P1227R2</a> ,<br><a href="#">P1970R2</a> | Free functions <a href="#">size</a> , <a href="#">ssize</a>                                                                                                                                                    | One is signed, the other isn't...                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <a href="#">P0463R1</a> ,<br><a href="#">P1612R1</a> | Endianness detection                                                                                                                                                                                           | An enum <a href="#">endian</a> whose members <a href="#">endian::native</a> , <a href="#">endian::little</a> , <a href="#">endian::big</a> describe the implementation's endianness.                                                                                                                                                                                                                                                                                                                                  |

| Document                                                                          | Summary                                                                  | Examples, notes                                                                                                                                                                                                                                                                                                                           |
|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">P0053R7</a> ,<br><a href="#">P0753R2</a>                              | Synchronized, buffered ostream                                           | A buffering ostream wrapper, <code>basic_ostream</code> , that causes all output to be emitted atomically to the wrapped ostream. Example: <code>std::ostream(std::cout) &lt;&lt; a &lt;&lt; b &lt;&lt; "foo" &lt;&lt; c &lt;&lt; '\n'</code> ; Unlike in the unwrapped version, concurrent writes to <code>cout</code> never interleave. |
| <a href="#">P0769R2</a>                                                           | <code>shift</code> algorithms                                            | New algorithms for shifting the elements of a range: <code>shift_left</code> , <code>shift_right</code>                                                                                                                                                                                                                                   |
| <a href="#">P0556R3</a> ,<br><a href="#">P1355R2</a> ,<br><a href="#">P1956R1</a> | Functions for integral power-of-2 operations                             | <code>has_single_bit</code> (i.e. is a power of two), <code>bit_ceil</code> , <code>bit_floor</code> , <code>bit_width</code>                                                                                                                                                                                                             |
| <a href="#">P0553R4</a>                                                           | Functions for bit operations                                             | Rotation ( <code>rotr</code> , <code>rotr</code> ), population count ( <code>popcount</code> ), count of number of successive zeros or ones from left or right ( <code>countl_zero</code> , <code>countl_one</code> , <code>countr_zero</code> , <code>countr_one</code> ).                                                               |
| <a href="#">P0811R3</a>                                                           | Functions <code>midpoint</code> , <code>lerp</code>                      | Interpolation functions. <code>std::midpoint(a, b)</code> computes essentially $a + b / 2$ , but works with integers, floating point numbers, and pointers. <code>std::lerp</code> computes arbitrary linear interpolation. Both are correct even in corner cases.                                                                        |
| <a href="#">P0586R2</a>                                                           | Function templates <code>cmp_equal</code> , <code>cmp_less</code> , etc. | These functions provide integral comparisons that produce the mathematical result even when the operands have different types. For example, <code>cmp_less(-1, 0U)</code> is <code>true</code> , whereas <code>-1 &lt; 0U</code> is <code>false</code> (due to arithmetic conversions).                                                   |
| <a href="#">P1020R1</a> ,<br><a href="#">P1973R1</a>                              | Function templates <code>make_unique_for_overwrite</code> ,              | The existing smart pointer creation functions <code>make_unique</code> ,                                                                                                                                                                                                                                                                  |

| Document                                             | Summary                                                                                | Examples, notes                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|------------------------------------------------------|----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                      | <code>make_shared_for_overwrite</code> ,<br><code>allocate_shared_for_overwrite</code> | <code>make_shared</code> and <code>allocate_shared</code> use value-initialization when called with no constructor arguments (equivalent to <code>new T()</code> ). This new feature adds functions <code>make_unique_for_overwrite</code> , <code>make_shared_for_overwrite</code> , and <code>allocate_shared_for_overwrite</code> , which take no constructor arguments and use default-initialization (equivalent to <code>new T</code> ). This avoids unnecessary initialization in situations where the initial value is never read. |
| <a href="#">P0356R5</a> ,<br><a href="#">P1651R0</a> | Function template <code>bind_front</code>                                              | A simpler, more predictable version of <code>bind</code> that only binds arguments on the left.                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <a href="#">P0325R4</a>                              | Function template <code>to_array</code>                                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <a href="#">P0653R2</a>                              | Function template <code>to_address</code>                                              | A well-defined replacement for <code>addressof(*p)</code> that works even when <code>p</code> does not point to a constructed object.                                                                                                                                                                                                                                                                                                                                                                                                      |
| <a href="#">P0591R4</a>                              | Utilities for the uses-allocator construction                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <a href="#">P1001R2</a>                              | New execution policy “unsequenced”                                                     | Unsequenced execution allows for vectorization (even on a single thread).                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |

## Modifications to existing features

| Document                                             | Summary                        | Examples, notes                                                                                                                                                                                                                                                                                                                            |
|------------------------------------------------------|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">P0482R6</a> ,<br><a href="#">P1423R3</a> | New type: <code>char8_t</code> | The type <code>char8_t</code> is analogous to the existing types <code>char16_t</code> and <code>char32_t</code> , i.e. a fundamental, unsigned integral type, sufficient to store one UTF-8 code unit. As a breaking change, the types of <code>u8</code> character literals (C++17, e.g. <code>u8'a'</code> ) and <code>u8</code> string |

| Document                                                                          | Summary                                                    | Examples, notes                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|-----------------------------------------------------------------------------------|------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                   |                                                            | <p>literals (C++11, e.g. <code>u8"foo"</code>) are now <code>char8_t</code> and array of <code>char8_t</code>, respectively; previously they were (array of) <code>char</code>. This breakage was deemed to be minor, since both of those features had not been widely used. This change enables overload resolution to distinguish intended UTF-8 from native encoding, which previously was only distinguished by the choice of literal, but not by the type of the literal value.</p>                                   |
| <a href="#">P1041R4</a>                                                           | Makes <code>char16_t/char32_t</code> literals be UTF-16/32 | <p>This requires that <code>char16_t</code> literals (e.g. <code>u"foo"</code>) be encoded as UTF-16 and <code>char32_t</code> literals (e.g. <code>U"foo"</code>) as UTF-32. This in analogy with <code>u8</code> literals (which are now <code>char8_t</code> literals), and which have already been required to be valid UTF-8.</p>                                                                                                                                                                                     |
| <a href="#">P0668R5</a> ,<br><a href="#">P0735R1</a> ,<br><a href="#">P0982R1</a> | Revising the memory model                                  | The memory model has been revised.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <a href="#">P0306R4</a> ,<br><a href="#">P1042R1</a>                              | Enhancements of variadic macros                            | <p>Variadic macros are now allowed to be invoked with zero variadic arguments (this was previously accepted as an extension by many compilers). Within a variadic macros, the new feature <code>__VA_OPT__(content)</code> can now be used, which expands to nothing if there are no variadic arguments, and to <code>content</code> otherwise. A typical use case is for a comma that is required if and only if there are variadic arguments: <code>define F(A, B, ...) vfunc(A, B __VA_OPT__(,) __VA_ARGS__)</code></p> |
| <a href="#">P0614R1</a>                                                           | Range-based <code>for</code> statements with initializer   | <p>A range-based <code>for</code> statement may now contain an additional initializer: <code>for (Data d = FetchData(); const Row&amp; row : d.rows()) { use(row); }</code></p>                                                                                                                                                                                                                                                                                                                                            |
| <a href="#">P0683R1</a>                                                           | Default member initializers for bit-fields                 | <p>Bit-fields can now have default member initializers: <code>struct X { int a : 2 {-2}; int b : 2 = 1; };</code></p>                                                                                                                                                                                                                                                                                                                                                                                                      |
| <a href="#">P0960R3</a> ,<br><a href="#">P1975R0</a>                              | Aggregate initialization from a parenthesized list         | <p>Aggregates can now be initialized with round parentheses instead of curly braces: <code>struct T { int a, b, c; }; T x(1, 2);</code> A (motivating) consequence is that <code>make_unique</code> and <code>emplace</code> now work with aggregate types.</p>                                                                                                                                                                                                                                                            |

| Document                                                                                                                                    | Summary                                                | Examples, notes                                                                                                                                                                                                                           |
|---------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">P0428R2</a>                                                                                                                     | Template head for generic lambdas                      | A generic lambda may now use a template head instead of (and in addition to) <code>auto</code> parameters, e.g.: <code>[]&lt;typename T&gt;(T a, T&amp; b, const T&amp; c) {}</code>                                                      |
| <a href="#">P0624R2</a>                                                                                                                     | Default-constructible and assignable stateless lambdas | Objects of stateless closure type (i.e. the type of a lambda without any captures) can now be default-constructed and assigned. This allows uses such as lambdas as container predicates.                                                 |
| <a href="#">P0315R4</a>                                                                                                                     | Lambdas in unevaluated contexts                        | Lambdas may now appear in unevaluated contexts (e.g. <code>using F = decltype([](int a, int b) { return a + b; }));</code> ). Combined with the above <a href="#">P0624R2</a> , this makes lambdas usable in a wider range of situations. |
| <a href="#">P0780R2</a> ,<br><a href="#">P2095R0</a> ,                                                                                      | Pack expansion in lambda <i>init-capture</i>           | An <i>init-capture</i> may now introduce a pack:<br><code>template &lt;typename ...Args&gt; void f(Args... args) { auto x = [...brgs = std::move(args)](){}; }</code>                                                                     |
| <a href="#">P0409R2</a>                                                                                                                     | Lambda capture [=, this]                               | It is now allowed to spell the lambda capture [=, <code>this</code> ] explicitly. Previously, the “ <code>this</code> ” was considered redundant and forbidden. See also <a href="#">P0806R2</a> above.                                   |
| <a href="#">P0961R1</a> ,<br><a href="#">P0969R0</a> ,<br><a href="#">P1091R3</a> ,<br><a href="#">P1381R1</a>                              | Improvements for structured bindings                   | <a href="#">P0961R1</a> and <a href="#">P0969R0</a> are intended as a defect resolution against C++17.                                                                                                                                    |
| <a href="#">P1009R2</a>                                                                                                                     | Array size deduction in <i>new-expressions</i>         | This allows <code>new int[]{1, 2, 3}</code> , which was previously ill-formed. This change is intended as a defect resolution against C++17.                                                                                              |
| <a href="#">P0702R1</a> ,<br><a href="#">P0739R0</a> ,<br><a href="#">P1814R0</a> ,<br><a href="#">P1816R0</a> ,<br><a href="#">P2082R1</a> | Class template argument deduction (“CTAD”)             | Various tweaks and improvements. Some of these are defect resolutions against C++17 ( <a href="#">P0702R1</a> , <a href="#">P0739R0</a> ). Aliases and aggregate templates can now use CTAD.                                              |
| <a href="#">P1825R0</a>                                                                                                                     | Copy elision and move semantics                        | Rvalue reference variables can now be moved implicitly in <code>return</code> statements. This change is intended as a defect resolution against C++17.                                                                                   |

| Document | Summary                                              | Examples, notes                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|----------|------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| P0634R3  | Fewer required <code>typename</code> disambiguations | This change removes the need to disambiguate a dependent name as a <code>typename</code> via the <code>typename</code> keyword from several places where this is already unambiguous. This removes some redundant ceremony from template code.                                                                                                                                                                                                                                                                |
| P0846R0  | ADL and function templates that are not visible      | This change fixes a long-standing, somewhat obscure situation, where it was not possible to provide explicit template arguments for a function template that is meant to be found through ADL when the name was not known to refer to a function template. For example, <code>f&lt;A&gt;(a)</code> would have been treated as a binary expression <code>f &lt; A</code> rather than as a template name <code>f&lt;A&gt;</code> . Previously, work-arounds were required to make ADL work in such a situation. |
| P0692R1  | Access checking on specializations                   | This change fixes a long-standing, somewhat obscure situation, where it was not possible to declare a template specialization for a template argument that is a private (or protected) member type. For example, given <code>class Foo { class Bar {}; };</code> , the access <code>Foo::Bar</code> is now allowed in <code>template&lt;class&gt; struct X; template&lt;&gt; struct X&lt;Foo::Bar&gt;;</code> .                                                                                               |
| P0892R2  | <code>explicit(bool)</code>                          | The <code>explicit</code> specifier now accepts an optional boolean parameter (where <code>explicit(true)</code> is equivalent to <code>explicit</code> and <code>explicit(false)</code> is equivalent to the absence of <code>explicit</code> ), which simplifies generic code where explicitness is “forwarded” from template parameters in some sense.                                                                                                                                                     |
| P1301R4  | An optional string for <code>[[nodiscard]]</code>    | A new form of the attribute now allows specifying a text (e.g. guidance for a user who is erroneously discarding a result), e.g. <code>[[nodiscard("did you mean to call a different function")]]</code> .                                                                                                                                                                                                                                                                                                    |
| P0848R3  | Conditionally trivial special member functions       | It is now possible to provide special member functions in both defaulted and non-defaulted forms at once, provided the declarations are constrained in a mutually exclusive way. This simplifies generic                                                                                                                                                                                                                                                                                                      |

| Document                                                                                                                                                | Summary                                                         | Examples, notes                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|---------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                         |                                                                 | code where defaultedness is “forwarded” from template parameters in some sense.                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| P1094R2                                                                                                                                                 | Nested inline namespaces                                        | Inline namespaces may now appear inside a nested namespace declaration: <code>namespace a::inline b::c {}</code>                                                                                                                                                                                                                                                                                                                                                                                                |
| P1002R1,<br>P1064R0,<br>P1327R1,<br>P1330R0,<br>P1331R2,<br>P1668R1                                                                                     | <code>constexpr</code> in the core language                     | Additional core language facilities are now allowed in <code>constexpr</code> functions: <code>try/catch</code> , virtual function calls, <code>dynamic_cast/typeid</code> , changing the active union member, trivial default initialization, <i>asm-definitions</i> .                                                                                                                                                                                                                                         |
| P0202R3,<br>P0415R1,<br>P0858R0,<br>P0879R0,<br>P0972R0,<br>P0980R1,<br>P1004R2,<br>P1006R1,<br>P1065R2,<br>P1023R0,<br>P1032R1,<br>P1424R1,<br>P1645R1 | <code>constexpr</code> in the library                           | Many more library functions (including member functions) are now <code>constexpr</code> : algorithms, utilities, <code>complex</code> , <code>swap</code> , <code>chrono</code> , <code>string</code> , <code>vector</code> , <code>pointer_traits</code> , <i>INVOKE</i> , array comparisons, <code>pair/tuple</code> , numeric algorithms. There is also a new notion of a <i>constexpr iterator</i> .                                                                                                        |
| P0600R1                                                                                                                                                 | <code>nodiscard</code> in the library                           | Several library (member) functions are now annotated with the <code>[[nodiscard]]</code> attribute (for example, <code>vector::empty</code> ).                                                                                                                                                                                                                                                                                                                                                                  |
| P0935R0                                                                                                                                                 | Remove accidentally- <code>explicit</code> default constructors | This large change replaces many explicit constructors that have default arguments, such as <code>explicit foo(Alloc a = Alloc());</code> , with two overloads, such that the resulting default constructor is not explicit. Previously, the one-parameter constructor was a default constructor, but because it was explicit, reasonable initializations such as <code>foo x = {};</code> had been ill-formed. This was unintended, since <code>explicit</code> was only meant to inhibit implicit conversions. |

| Document                                             | Summary                                                                             | Examples, notes                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|------------------------------------------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">P0919R3</a> ,<br><a href="#">P1690R1</a> | Heterogeneous lookup for unordered containers                                       | Analogous to heterogeneous lookup for ordered containers in C++14, this change allows transparent predicates to be defined and used in unordered containers. For example, an <code>unordered_map</code> with key <code>string</code> that uses transparent predicates can look up a <code>string_view</code> or a <code>const char*</code> without constructing a <code>string</code> object for the lookup. The details are more complicated than for ordered containers, since equality and hashing need to be compatible even when the types differ. |
| <a href="#">P0458R2</a>                              | Element existence in associative containers (“contains”)                            | A new member function, <code>m.contains(key)</code> , equivalent to <code>m.find(key) != m.end()</code> , for all associative containers.                                                                                                                                                                                                                                                                                                                                                                                                               |
| <a href="#">P0646R1</a>                              | Return element counts from <code>list</code> , <code>forward_list</code> operations | The modifying member functions <code>remove</code> , <code>remove_if</code> and <code>unique</code> of <code>list</code> and <code>forward_list</code> now return the number of removed elements.                                                                                                                                                                                                                                                                                                                                                       |
| <a href="#">P0457R2</a>                              | Prefix/suffix checking for strings                                                  | The class templates <code>basic_string_view</code> and <code>basic_string</code> have new member functions <code>starts_with</code> and <code>ends_with</code> that test whether the string starts/ends with the given prefix/suffix (string or single character).                                                                                                                                                                                                                                                                                      |
| <a href="#">P0674R1</a>                              | <code>make_shared</code> for arrays                                                 | The <code>make_shared</code> creation function can now be used for array types, e.g: <code>make_shared&lt;T[]&gt;(N)</code> ; (Array types are supported by <code>shared_ptr</code> since C++17.)                                                                                                                                                                                                                                                                                                                                                       |
| <a href="#">P0020R6</a>                              | Floating-point atomic                                                               | <code>atomic&lt;float&gt;</code> , <code>atomic&lt;double&gt;</code> , <code>atomic&lt;long double&gt;</code>                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| <a href="#">P0718R2</a>                              | Atomic <code>shared_ptr</code> and <code>weak_ptr</code>                            | There are now partial specializations <code>atomic&lt;shared_ptr&lt;T&gt;&gt;</code> and <code>atomic&lt;weak_ptr&lt;T&gt;&gt;</code> that allow atomic access to a shared or weak pointer. This replaces the existing free function overloads <code>shared_ptr&lt;T&gt; atomic_load(const shared_ptr&lt;T&gt;*)</code> etc., which were fragile and error-prone. The old free function overloads are now deprecated.                                                                                                                                   |

| Document                                             | Summary                                                               | Examples, notes                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|------------------------------------------------------|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">P0528R3</a> ,<br><a href="#">P1123R0</a> | Atomic compare-and-exchange in the presence of padding                | This change makes it a requirement that compare-and-exchange for an atomic type <code>atomic&lt;T&gt;</code> works even when the type <code>T</code> contains padding bits. That is, whether the expected value equals the current value is no longer allowed to depend on unspecified padding; implementations may need to do extra work to make this happen (e.g. always store a predictable bit pattern in the padding, or use a masked comparison). |
| <a href="#">P0439R0</a>                              | Make <code>memory_order</code> a scoped enumeration                   | Instead of <code>std::memory_order_relaxed</code> we now have <code>std::memory_order::relaxed</code> , etc.. The old names are provided as free constants for compatibility.                                                                                                                                                                                                                                                                           |
| <a href="#">P0616R0</a>                              | Use <code>std::move</code> in <code>&lt;numeric&gt;</code> algorithms | Algorithms such as <code>accumulate</code> used to be specified as having the binary operation called like <code>acc = op(acc, *it)</code> . This has now changed to <code>acc = op(std::move(acc), *it)</code> .                                                                                                                                                                                                                                       |
| <a href="#">P0408R7</a>                              | Efficient access to the buffer of <code>basic_stringbuf</code>        | API additions to <code>basic_stringbuf</code> and string-stream classes that allow moving the buffering string in and out.                                                                                                                                                                                                                                                                                                                              |
| <a href="#">P1148R0</a>                              | Comprehensive review of [strings]                                     | A thorough cleanup of the specification of <code>char_traits</code> , <code>basic_string_view</code> , and <code>basic_string</code> . This resolves numerous LWG issues.                                                                                                                                                                                                                                                                               |

## Miscellaneous

| Document                                                                          | Summary                                 | Examples, notes                                                                                                                                                                                                                                                                                                                                                       |
|-----------------------------------------------------------------------------------|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <a href="#">P0941R2</a> ,<br><a href="#">P1353R0</a> ,<br><a href="#">P1902R1</a> | Feature test macros                     | Feature test macros are now part of the working paper. This is not relevant to the published International Standard, but it enables vendors to support <i>upcoming</i> language revisions incrementally and inform users which features are provided by the implementation. The feature test macros were previously maintained in the separate <a href="#">SD-6</a> . |
| <a href="#">P0754R2</a>                                                           | The <code>&lt;version&gt;</code> header | The new header <code>&lt;version&gt;</code> is provided. It is formally empty and not relevant to the published International                                                                                                                                                                                                                                         |

| Document                                                                                                                                                                                     | Summary                                                                                                                         | Examples, notes                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                                                                                                                                              |                                                                                                                                 | Standard, but it provides a standardized place for implementation-defined information. Previously, the now-removed header <code>&lt;ciso646&gt;</code> (see P0619R4 above) was frequently and informally used for this purpose. The new feature test macros (see P0941R2 above) are defined in this header (as well as in their contextually appropriate place).                                                                                                                                                                                                                                                                                            |
| P0788R3;<br>P1458R1,<br>P1459R1,<br>P1460R1,<br>P1462R1,<br>P1463R1,<br>P1464R1,<br>P1622R3,<br>P1686R2,<br>P1718R2,<br>P1719R2,<br>P1720R2,<br>P1721R2,<br>P1722R2,<br>P1723R2,<br>P2045R1. | Replace<br><i>Requires</i> with<br><i>Constraints</i> ,<br><i>Mandates</i> ,<br><i>Preconditions</i><br>in the<br>specification | A change to the formal specification of the library only: the old <i>Requires</i> element, which was serving multiple roles, is now obsolete and replaced by three new elements, <i>Constraints</i> (statically checked, affects overload resolution), <i>Expects</i> (preconditions whose violation results in undefined behaviour), and <i>Mandates</i> (statically checked conditions whose violation makes the program ill-formed). This should not have any observable effect, but it makes it clearer which requirements are diagnosed, which are the user's responsibility, and which affect overload resolution (but do not produce errors per se). |
| P0551R3                                                                                                                                                                                      | Restrict use of<br><code>std</code> functions<br>and function<br>templates                                                      | A significant restriction in what users are allowed to do with standard library functions and function templates: Users are no longer allowed to specialize standard library function templates, and in general to take the address of a standard library function (unless the function is explicitly specified as <i>addressable</i> ). This improves implementer freedom greatly to, say, implement a function as an overload set or a template. It marks a significant clarification of policy: functions in the standard library are in general only supposed to be understood as <i>valid call expressions</i> , not literally as declared functions.  |
| P1025R1                                                                                                                                                                                      | Update<br>reference to<br>Unicode                                                                                               | Adds an undated reference to ISO/IEC 10646, and replaces "UCS4" with "UTF-32".                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |

## Unlisted papers

The following papers were moved at committee meetings, but their contents are too specific to call out as separate features: [P0339R6](#), [P0340R3](#), [P0357R3](#), [P0388R4](#), [P0475R1](#), [P0487R1](#), [P0602R4](#), [P0608R3](#), [P0641R2](#), [P0655R1](#), [P0704R1](#), [P0738R2](#), [P0759R1](#), [P0771R1](#), [P0809R0](#), [P0883R2](#), [P0899R1](#), [P1008R1](#), [P1131R2](#), [P1139R2](#), [P1165R1](#), [P1207R4](#), [P1285R0](#), [P1522R1](#), [P1638R1](#), [P1908R1](#), [P1960R0](#), [P1961R0](#), [P1963R0](#), [P1965R0](#), [P2101R0](#). Additionally, the following otherwise unlisted papers were accepted as defect resolutions against C++17: [P0588R1](#), [P0593R6](#), [P0682R1](#), [P0727R0](#), [P0859R0](#), [P0929R2](#), [P0962R1](#), [P1164R1](#), [P1286R2](#), [P1771R1](#), [P1957R2](#).

The following papers contain issues that have been accepted as defect reports. CWG issues are handled by [P0710R1](#), [P0711R0](#), [P0817R0](#), [P0818R1](#), [P0968R0](#), [P1113R0](#), [P1114R0](#), [P1350R0](#), [P1358R0](#), [P1359R0](#), [P1510R0](#), [P1969R0](#), [P1968R0](#), [P1971R0](#), [P2103R0](#), [P2108R0](#). LWG issues are handled by [P0698R0](#), [P0699R0](#), [P0815R0](#), [P0864R0](#), [P0888R0](#), [P1003R0](#), [P1082R0](#), [P1224R0](#), [P1457R0](#), [P1724R0](#), [P1917R0](#), [P2051R0](#), [P2117R0](#). Only specific issues may have been selected from each paper; the meeting minutes contain the details. The complete accumulation of accepted issues is published in the regularly updated [CWG issues list](#) and [LWG issues list](#).

## Defects, issues, bug fixes

The committee maintains a list of *issues* (see links above) which contain suggestions for improvements in the normative wording (clarifications, simplifications, corrections, etc.). When an issue is accepted that affects the published standard (not just the new work in progress), we have a *defect report*. The complete specification of C++ consists of the latest published document as amended by the resolutions of all subsequent defect reports. The resolution of a defect report is also incorporated into the working paper (as applicable), and thus the next published revision of the Standard no longer has that particular issue.

Therefore, the answer to the question of, say, “what is a valid C++14 program?” changes over time, up until the publication of C++17, and so forth. In practice, the situation is a bit more complicated when compiler vendors offer conformance modes for specific language revisions (e.g. `-std=c++11`, `-std=c++17`). Vendors may consider defect resolutions to apply to *any* historic revision that contains the defect (whereas ISO considers only the most recent publication as *the* Standard).

Papers that have been applied to the working draft since the publication of C++20 and that are intended as defect reports are annotated with “DR” in the above change summary. Defect reports against C++20 that concern older (pre-C++20) content are not called out here specifically, nor are defects against C++17 whose resolutions are included in C++20. The full history of accepted issues together with the Standard revisions to which they apply is implicit in the CWG and LWG issues lists, but it is difficult to present explicitly.

## Assorted snippets demonstrating C++20

```
int main() {}
```